Microservices System Design

Interview Study Guide

Design • Security • OAuth 2.0 with PKCE

Prepared for

Sachin Joshi • Senior Architect

July 2026

> *A revision-ready reference covering end-to-end microservices design, the full security surface of distributed systems, and the OAuth 2.0 Authorization Code + PKCE flow — each paired with likely interview questions and answer pointers.*

Contents

> *How to use this guide: read the considerations for the concepts, then drill the question banks aloud. The 'How to sound senior' notes flag the signals interviewers reward.*

Part 1 • Microservices: End-to-End Design

1.1 The End-to-End Framework

When handed a design prompt (“design an order / ride-sharing / checkout system”), walk through these phases aloud. The final row — explicit trade-offs — is what separates a senior answer from a mid-level one.

Phase	What to cover
Requirements	Functional + non-functional (SLA, RPS, latency p99, data volume, consistency needs)
Capacity estimation	QPS, storage, bandwidth, read/write ratio → sizing
Service decomposition	Bounded contexts, ownership, granularity
API & contracts	Sync vs async, gateway, versioning
Data	DB-per-service, consistency model, transactions
Communication	Messaging, event-driven, orchestration vs choreography
Cross-cutting	Resilience, security, observability, configuration
Deployment & scale	Containers, orchestration, autoscaling, rollout strategy
Trade-offs	Name what you optimised for and what you sacrificed

1.2 Service Decomposition

- **Domain-Driven Design:** align services to business capabilities and bounded contexts, not technical layers.
- **Granularity:** avoid nano-services (chatty, high ops overhead) and mini-monoliths. A service owns one capability plus its data.
- **Ownership:** one team owns a service end to end — “you build it, you run it”.
- **Coupling test:** if two services always change or deploy together, they may belong together.

1.3 Inter-Service Communication

- **Synchronous (REST / gRPC):** simple mental model, but creates temporal coupling and cascading failure risk. Use gRPC for low-latency, high-throughput internal calls.
- **Asynchronous (Kafka / RabbitMQ / Service Bus):** decouples producers and consumers, absorbs load spikes, enables event-driven flows.
- **Rule of thumb:** queries → sync; state-change propagation and workflows → async events.

1.4 Data Management (the hardest part)

- **Database per service:** each service owns its schema. A shared DB is a distributed monolith.

- **Distributed transactions** → **Saga:** choreography (events, no coordinator) vs orchestration (a coordinator drives steps). Each step has a compensating transaction for rollback.
- **CQRS:** separate read and write models when their access patterns diverge.
- **Event Sourcing:** store state as an append-only event log for audit, replay, temporal queries — adds complexity, use where it earns its keep.
- **Outbox pattern (+ CDC / Debezium):** atomically persist state and publish events, avoiding dual-write inconsistency. Embrace eventual consistency.

1.5 API Gateway & Service Discovery

- **API Gateway:** single entry point for routing, auth, rate limiting, aggregation, TLS termination. Watch for it becoming a bottleneck or god-object.
- **BFF (Backend-for-Frontend):** tailored gateways per client (web / mobile).
- **Service discovery:** client-side (Eureka) or server-side (Kubernetes DNS + Service).

1.6 Resilience & Fault Tolerance

- **Circuit breaker** — stop calling a failing dependency.
- **Retries** with exponential backoff + jitter — but make the operation idempotent first.
- **Timeouts** on every network call — never wait indefinitely.
- **Bulkhead** — isolate resource pools so one slow dependency does not exhaust threads.
- **Rate limiting, fallbacks, graceful degradation,** and **idempotency keys** for safe write retries.

1.7 Observability (three pillars)

Pillar	Tooling & practice
Logging	Structured, centralised (ELK / Loki), with correlation IDs
Metrics	RED (Rate, Errors, Duration) / USE — Prometheus + Grafana
Tracing	Distributed tracing across hops — OpenTelemetry + Jaeger / Zipkin

1.8 Deployment, Scale & Security

- **Containers + Kubernetes:** self-healing, autoscaling (HPA), rolling updates.
- **Progressive delivery:** blue-green, canary, feature flags to de-risk releases.
- **Stateless services** → horizontal scale; cache at CDN / gateway / Redis; scale DBs via replicas and sharding.
- **Security** is a full layer of its own — see Part 2.

1.9 Interview Questions

Conceptual

1 Monolith vs microservices — when would you **not** use microservices? *(Small team, unclear domain, early stage; ops overhead outweighs benefits. Start monolith, extract later.)*

2. How do you decide service boundaries? *(DDD bounded contexts, business capabilities, data ownership, team topology.)*
3. What is a “distributed monolith” and how do you avoid it? *(Shared DB, sync chains, lock-step deploys → independent data + async decoupling.)*

Data & consistency

1. How do you handle a transaction spanning three services? *(Saga with compensations, not 2PC.)*
2. How do you keep a DB write and an event publish atomic? *(Transactional Outbox + CDC.)*
3. Explain eventual consistency and how you would surface stale data to users.
4. When would you use CQRS / Event Sourcing — and when not?

Communication & resilience

1. Sync vs async — trade-offs? *(Coupling, latency, failure isolation, complexity.)*
2. How do you prevent cascading failures? *(Circuit breaker, timeouts, bulkhead, load shedding.)*
3. How do you make retries safe? *(Idempotency keys; at-least-once vs exactly-once semantics.)*

Operations & scenario

1. How do you trace a request across ten services? *(Correlation IDs + distributed tracing.)*
2. How do you deploy with zero downtime? *(Rolling / blue-green / canary + backward-compatible APIs.)*
3. How do you version APIs without breaking consumers? *(Additive changes, consumer-driven contract testing / Pact.)*
4. Design the e-commerce checkout flow (order, payment, inventory, notification). *(Saga, idempotency, inventory reservation.)*
5. How would you handle a Black Friday 10x spike? *(Autoscaling, caching, async queues, rate limiting, graceful degradation.)*

> ***Senior signals:*** *quantify (capacity, SLAs, p99); name trade-offs against requirements; raise failure modes and operability unprompted; know when not to use a pattern; set consistency requirements per operation rather than one model globally.*

Part 2 • Security Considerations

A distributed system is harder to secure than a monolith: in-process calls become network hops, one deployable becomes dozens, and a single trust boundary becomes many. Lead with **defence in depth** and **zero trust**.

2.1 Authentication & Authorization

- **Edge authentication:** authenticate once at the gateway (OAuth2 / OIDC), issue a token, propagate identity downstream.
- **Token type:** JWT (stateless, no lookup, hard to revoke) vs opaque/reference tokens (revocable, needs introspection). Trade-off = revocability vs latency.
- **AuthZ placement:** coarse-grained at the gateway; fine-grained business-rule authZ inside each service. Externalise complex policy with OPA.
- **Service identity:** every service is its own principal (SPIFFE/SPIRE, mTLS certs, service accounts). Internal calls are not implicitly trusted.
- **Delegation:** client_credentials for service-to-service; authorization_code for users; token exchange for on-behalf-of.

2.2 Service-to-Service (East-West) Security

- **Zero trust internally:** never assume the private network is safe. Every hop is authenticated and encrypted.
- **mTLS everywhere:** both sides present certs. A service mesh (Istio / Linkerd) automates issuance, rotation and enforcement without app changes.
- **Network policies:** Kubernetes NetworkPolicies / segmentation — a compromised service reaches only what it needs.
- **Short-lived certs** (SPIFFE identities) so rotation is routine, not a fire drill.

2.3 API Gateway & Edge Security

- Gateway owns TLS termination, authN, rate limiting, throttling, request validation, WAF, IP allow/deny.
- Rate limiting and quotas blunt DDoS and abuse (per-user, per-IP, per-endpoint).
- Validate schema at the edge — but keep defence in depth; services still validate.

2.4 Secrets & Configuration

- Never hardcode secrets in code, images, or committed env/config files.
- Central secrets manager (Vault, cloud KMS) with dynamic, short-lived, auto-rotated secrets.
- Encryption at rest and in transit; key rotation, envelope encryption, HSM-backed keys.
- Scan images and repos for leaked secrets (git-secrets, trufflehog).

2.5 Data Security & Privacy

- Encrypt sensitive data at rest; field-level for PII / PCI / PHI.

- Owning service is the single authority; others get data via APIs/events, not direct DB reads.
- Minimise sensitive data on the event bus — prefer references/tokens over raw PII.
- Compliance: GDPR right-to-erasure (hard with event sourcing → crypto-shredding), PCI-DSS scope isolation, data residency.

2.6 Supply Chain & Runtime Security

- Minimal base images, CVE scanning (Trivy / Snyk), signed images (Cosign / Sigstore), non-root containers.
- Dependency SCA scanning, SBOM, pinned versions, patch cadence.
- Harden CI/CD — it is a high-value target: no secrets in logs, protected branches, artifact signing.
- Runtime: pod security standards, read-only filesystems, admission controllers, threat detection (Falco).

2.7 Observability & Resilience for Security

- Immutable, tamper-evident audit logs (who did what, when); correlation IDs to trace an attack across services.
- Centralised security monitoring / SIEM, anomaly detection, alerting.
- Never log secrets, tokens, or full PII — a common breach vector.
- Rate limiting, circuit breakers and idempotency double as abuse / DoS / replay defences.

2.8 Threat Modelling — STRIDE in Microservices

Structuring a design answer with STRIDE is a strong senior signal.

Threat	Microservices control
Spoofing	mTLS, service identity, strong authN
Tampering	TLS in transit, signed messages, integrity checks
Repudiation	Audit logs, correlation IDs
Information disclosure	Encryption, least privilege, data minimisation
Denial of service	Rate limiting, autoscaling, circuit breakers
Elevation of privilege	Fine-grained authZ, least-privilege accounts, network policies

2.9 Interview Questions

AuthN / AuthZ & identity

1. Where do you authenticate a user, and how does identity flow downstream?
2. JWT vs opaque tokens — trade-offs? How do you revoke a JWT before it expires? *(Short TTL + refresh, denylist, key rotation, reference tokens.)*
3. Coarse vs fine-grained authorization — where does each belong?
4. What is the “confused deputy” problem and how do you prevent it?
5. How does Service A prove its identity to Service B?

Network, secrets & data

1. Explain zero trust here — why isn't a private VPC "secure enough"?
2. How does mTLS work and what does a service mesh add?
3. A service is compromised — how do you limit lateral movement? *(Segmentation, least privilege, short-lived creds.)*
4. How do you manage secrets across dozens of services and environments?
5. Handling PII across many services and an event log — and GDPR right-to-be-forgotten with event sourcing? *(Crypto-shredding, tombstones, forgettable payloads.)*

Supply chain, ops & applied

1. How do you secure the software supply chain for containers? What is an SBOM?
2. Your CI/CD pipeline is a target — how do you harden it?
3. How would you detect and investigate an incident spanning multiple services?
4. How do resilience patterns double as security controls? How do you prevent replay attacks on a payment API? *(Idempotency keys, nonces, request signing, short-lived tokens.)*

> ***Sound senior:*** *distinguish edge (north-south) from internal (east-west) security; raise the JWT revocation problem unprompted; tie controls to compliance and blast-radius containment; acknowledge trade-offs (mTLS latency/ops cost, statelessness vs revocability).*

Part 3 • OAuth 2.0 Authorization Code Flow with PKCE

3.1 Why PKCE Exists

The classic Authorization Code flow uses a **client_secret** to prove the client's identity when exchanging the code for a token. That works for **confidential clients** (backends that can keep a secret). **Public clients** — SPAs, mobile and desktop apps — cannot: the code ships to the user's device and can be inspected.

PKCE (Proof Key for Code Exchange, RFC 7636) replaces the static secret with a dynamically generated, per-request secret. It defends against the **authorization code interception attack**: even if a malicious app intercepts the code, it cannot exchange it without the code_verifier.

> *PKCE is now recommended for **all** clients — public and confidential (defence in depth). The Implicit flow is deprecated; OAuth 2.1 removes it.*

3.2 The Actors

Role	Example
Resource Owner	The end user
Client	The app (SPA / mobile) requesting access
Authorization Server	Issues tokens (Entra ID, Auth0, Okta, Keycloak)
Resource Server	The API holding protected data

3.3 The Two PKCE Secrets

Before the flow starts, the client generates:

- **code_verifier** — a high-entropy random string (43–128 chars).
- **code_challenge** — derived from it. **S256** (recommended): code_challenge = BASE64URL(SHA256(code_verifier)). **plain** only if S256 is unsupported.

The verifier stays on the device; only the hashed challenge crosses the front channel. SHA-256 is one-way, so an interceptor who sees the challenge cannot reverse it to the verifier.

3.4 End-to-End Flow

Step-by-step

1. Client prepares PKCE: generates code_verifier, computes code_challenge.
2. Authorization request (browser redirect) sends code_challenge + method, plus state (CSRF) and nonce (OIDC id_token binding).
3. User authenticates and consents at the AS.
4. AS redirects back with a short-lived, single-use authorization code; it stored the challenge with the code.
5. Token exchange (back channel, direct HTTPS POST) sends the code plus the original code_verifier.
6. AS hashes the verifier and compares it to the stored challenge — the redeemer must be the initiator.
7. AS returns access_token, id_token (OIDC), and optionally refresh_token.
8. Client calls the API with the bearer access token; the resource server validates signature and claims and returns data.

3.5 The Tokens

Token	Purpose	Notes
access_token	Authorises API calls	Short-lived; often JWT; validate signature/claims. AuthoriZation.
id_token (OIDC)	Proves who the user is	JWT of user claims. AuthenticatioN. Never send to APIs.
refresh_token	Silently gets new access tokens	Long-lived, high value. Use rotating refresh tokens.

> ***Common confusion:*** *OAuth2 = authorization (delegated access). OIDC (built on top) = authentication (identity via id_token). PKCE secures the code flow; OIDC adds the id_token / nonce.*

3.6 Considerations

- PKCE is **not** a replacement for client authentication — confidential clients use PKCE *and* the secret.
- Always use **S256**, never plain.
- **state** is a mandatory CSRF defence; the client verifies it round-trips unchanged. **nonce** binds the id_token to the session.
- **Redirect URI** must be pre-registered and exact-matched (no wildcards).
- Authorization code is single-use and short-lived (~30–60s); the AS rejects reuse.
- **Token storage in SPAs:** avoid localStorage (XSS). Prefer in-memory + rotating refresh, or the BFF pattern (server holds tokens; browser gets an HttpOnly, Secure, SameSite cookie).
- **Refresh token rotation:** issue a new token each use, revoke the old, detect reuse.
- **Resource server validation:** verify signature (JWKS), iss, aud, exp, nbf, scopes — do not merely decode.
- HTTPS everywhere; request minimum scopes; scope tokens to a specific API via audience/resource.

3.7 Interview Questions

Conceptual & flow

1. What is PKCE and what specific attack does it prevent? *(Auth code interception on public clients.)*
2. Why can't a SPA or mobile app use a client secret?
3. Walk through the Authorization Code + PKCE flow end to end.
4. code_verifier vs code_challenge — which is sent when? *(Challenge in auth request; verifier in token request.)*
5. Why is S256 preferred over plain?

OAuth vs OIDC & tokens

1. OAuth2 vs OpenID Connect — what does each solve? *(Authorization vs authentication.)*
2. access_token vs id_token vs refresh_token — purpose of each?
3. Can you use an id_token to call an API? Why not? *(Wrong audience; it is authN proof, not scoped authZ.)*
4. How does a resource server validate a JWT access token? *(JWKS signature, iss/aud/exp/scope.)*

Security depth & applied

1. What does state defend against, and how? What is the role of nonce?
2. How do you securely store tokens in a browser? *(In-memory + rotating refresh, or BFF with HttpOnly cookies.)*
3. If an attacker intercepts the code, why can't they get a token under PKCE? *(No verifier; hash won't match.)*
4. Why is the Implicit flow deprecated? *(Tokens in URL fragment, no PKCE, exposure via history / referrer / logs.)*
5. Design SSO for a SPA + mobile app + backend API — which flows for which client? *(Auth Code + PKCE for SPA & mobile; client_credentials for service-to-service.)*

6 What changes in OAuth 2.1? *(PKCE mandatory for all code flows, Implicit + Password grants removed, exact redirect-URI matching, refresh-token rotation for public clients.)*

> ***Sound senior:*** *define PKCE as “proof that the code redeemer is the initiator”; volunteer the OAuth-vs-OIDC / authZ-vs-authN distinction; raise SPA token storage and the BFF pattern unprompted; mention OAuth 2.1 and universal PKCE; tie everything back to defence in depth.*